PRODUCT BACKLOG

Projet

Bank Data Pipeline Medallion

Document pédagogique — Architecture de données bancaires

🥉 BRONZE	🥈 SILVER	🥇 GOLD
Ingestion brute	Nettoyage & qualité	Analytics & KPIs

Version 2.0 — Document enrichi à visée éducative

1. Avant-propos & Objectifs pédagogiques

Ce document constitue le Product Backlog enrichi du projet « Bank Data Pipeline Medallion ». Au-delà d'une simple liste de tâches, il est conçu comme un guide d'apprentissage structuré pour un Data Engineer en formation souhaitant maîtriser les fondamentaux de l'ingénierie de données appliquée au secteur bancaire.

Product Goal

Bâtir un pipeline de données automatisé et scalable pour une banque internationale, capable de transformer des transactions brutes en indicateurs financiers stratégiques, en maîtrisant chaque étape technique sans dépendance aux IA de génération de code.

Compétences visées à l'issue du projet

Domaine	Compétences acquises
Python	Génération de données, manipulation Pandas/PyArrow, scripts d'automatisation
Architecture	Comprendre et implémenter le pattern Medallion (Bronze / Silver / Gold)
Data Quality	Mettre en œuvre des contrôles de qualité (nulls, doublons, anomalies)
Sécurité	Anonymisation RGPD, pseudonymisation des données sensibles
Modélisation	Construire un schéma en étoile (Faits + Dimensions)
Industrialisation	Orchestrer un pipeline avec Apache Airflow ou cron avancé
Collaboration	Utiliser Git, Docker, structurer un projet professionnel

2. L'Architecture Medallion — Fondement du projet

L'architecture Medallion est un pattern de conception de datalake organisé en trois couches progressives. Chaque couche représente un niveau de qualité et de transformation de la donnée croissant. Ce modèle est notamment popularisé par Databricks et est aujourd'hui considéré comme une best practice de l'industrie.

BRONZE Couche d'ingestion brute	**SILVER** Nettoyage & normalisation	**GOLD** Business logic & Analytics
• Données brutes non modifiées • Fichiers CSV / JSON d'origine • Colonnes techniques ajoutées • Format Parquet • Historique complet conservé	• Doublons supprimés • Nulls gérés • Formats standardisés • Données RGPD anonymisées • Prête pour l'analyse	• Agrégations métier • KPIs calculés • Schéma en étoile • Consommable par BI • Tables Faits + Dimensions

Principe clé : l'immutabilité

Dans l'architecture Medallion, on ne modifie JAMAIS les données d'une couche inférieure. Si une correction est nécessaire, on rejoue la transformation. Cela garantit la traçabilité complète et la reproductibilité des résultats — une exigence absolue en environnement bancaire.

ETL vs ELT — Quel paradigme choisir ?

Critère	ETL (Extract-Transform-Load)	ELT (Extract-Load-Transform)
Ordre des opérations	Transformation avant chargement	Chargement d'abord, transformation après
Lieu de transformation	Outil externe (ex: script Python)	Directement dans le data warehouse
Adapté pour	Volumes modérés, règles complexes	Big Data, cloud warehouses (Snowflake, BigQuery)
Avantage principal	Contrôle fin des transformations	Scalabilité, flexibilité
Choix pour ce projet	✔ Recommandé (scripts Python maîtrisés)	Option avancée possible avec DuckDB

Pour ce projet pédagogique, l'approche ETL est privilégiée : elle permet de voir explicitement chaque étape de transformation dans le code Python, favorisant ainsi la compréhension.

3. Les Épics & User Stories détaillées

Les 5 épics ci-dessous couvrent le cycle complet du projet, du poste de travail jusqu'à l'industrialisation. Chaque User Story est décrite sous forme de carte avec son récit, ses critères d'acceptation et les concepts techniques pédagogiques associés.

ÉPIC 1 — FONDATIONS & ENVIRONNEMENT | Le Socle

Objectif : Préparer le terrain de jeu avant d'écrire la première ligne de code métier. Un environnement mal configuré est la première source de blocages en projet data. Cette phase, souvent sous-estimée, est pourtant critique.

US.01 Comprendre l'architecture Medallion	
Récit	En tant que **Data Engineer** Je veux *comprendre l'architecture Medallion (Bronze, Silver, Gold)* Afin de savoir comment organiser mes données de manière professionnelle et maintenable
Critères	✔ Je peux expliquer avec mes mots les 3 couches et leur rôle respectif ✔ Je sais quelle transformation appartient à chaque couche ✔ Je comprends pourquoi on ne modifie pas les données en Bronze ✔ Je connais au moins 3 différences entre l'architecture Medallion et un simple dossier de fichiers
Concepts clés	• Datalake vs Data Warehouse : deux paradigmes complémentaires • Format Parquet : stockage colonnaire, compression efficace, compatible Python/Spark • Partitionnement temporel : organiser les fichiers par date (YYYY/MM/DD) pour les perfs • Idempotence : une transformation rejouée donne toujours le même résultat • Schema-on-read vs Schema-on-write : flexibilité versus rigueur

US.02 Installer l'environnement de développement	
Récit	En tant que **Dev** Je veux *installer l'environnement de travail complet (Python, VS Code, Git, Docker)* Afin de collaborer efficacement avec mon binôme et garantir la reproductibilité du projet
Critères	✔ Python 3.10+ installé avec un environnement virtuel (venv) fonctionnel ✔ VS Code configuré avec les extensions Python, Docker, GitLens ✔ Un dépôt Git créé avec une structure de dossiers conforme (bronze/, silver/, gold/) ✔ Docker Desktop installé et un docker-compose.yml minimal fonctionnel ✔ Un README.md décrit les étapes d'installation pour n'importe quel collaborateur

Concepts clés	• Environnements virtuels Python (venv / conda) : isoler les dépendances par projet • Git flow : branches main / develop / feature — convention de commits (Conventional Commits) • .gitignore : ne jamais versionner les données brutes ni les secrets (.env) • Docker : notion d'image, de conteneur, de volume — pourquoi containeriser ? • requirements.txt vs pyproject.toml : gestion des dépendances Python • Variables d'environnement : sécuriser les credentials via un fichier .env

US.03 Comprendre ETL vs ELT	
Récit	En tant que **Dev** Je veux *comprendre la différence fondamentale entre ETL et ELT* Afin de choisir l'approche la plus adaptée à un contexte bancaire avec des contraintes de conformité
Critères	✔ Je peux donner un exemple concret de cas d'usage ETL vs ELT ✔ Je comprends pourquoi les banques préfèrent historiquement l'ETL (auditabilité) ✔ Je suis capable d'identifier à quelle étape se fait la transformation dans mon pipeline ✔ Un schéma comparatif est produit et intégré dans la documentation
Concepts clés	• ETL (Extract → Transform → Load) : la transformation a lieu AVANT le chargement • ELT (Extract → Load → Transform) : on charge d'abord les données brutes, puis on transforme • OLTP vs OLAP : systèmes transactionnels vs analytiques — des besoins différents • Latence vs throughput : trade-offs selon le volume de données • Outils ETL courants : Talend, Informatica, Apache NiFi — vs ELT : dbt, BigQuery, Snowflake

US.04 Définir le schéma des données sources	
Récit	En tant que **Product Owner** Je veux *définir le schéma précis des données sources (Transactions, Clients, Agences)* Afin de simuler une activité bancaire réelle avec des données cohérentes et représentatives
Critères	✔ Un dictionnaire de données est produit pour les 3 entités (Transactions, Clients, Agences) ✔ Chaque champ est documenté : nom, type, format, contraintes, exemple de valeur ✔ Les relations entre entités sont identifiées (clés primaires / étrangères) ✔ Les cas de données "sales" sont documentés (valeurs manquantes, doublons potentiels) ✔ Le volume cible est défini : 100 000 transactions/jour, cohérence des

	distributions
Concepts clés	• Modélisation relationnelle : entités, attributs, cardinalités (1-N, N-N) • Clé primaire vs clé étrangère : intégrité référentielle des données • Types de données bancaires : IBAN, BIC, devise ISO 4217, montants en centimes (int vs float) • Données personnelles RGPD : identifier et catégoriser dès la conception (Privacy by Design) • Distribution statistique des montants : loi log-normale pour simuler des transactions réalistes

ÉPIC 2 — COUCHE BRONZE | Ingestion — Raw Data

Objectif : Récupérer la donnée brute et la stocker de manière sécurisée et traçable. Le mantra de la couche Bronze : « Ne jamais perdre une donnée brute. » Toute erreur de transformation doit pouvoir être rejouée à partir de la source.

US.05 Générer 100 000 transactions bancaires simulées

Récit	En tant que **Dev** Je veux *écrire un script Python qui génère des fichiers CSV/JSON simulant 100 000 transactions bancaires par jour* Afin de disposer d'un jeu de données réaliste pour tester toutes les couches du pipeline
Critères	✔️ Le script generate_data.py fonctionne avec un paramètre --date YYYY-MM-DD ✔️ 100 000 transactions générées avec des distributions réalistes des montants ✔️ Au moins 2% de données « sales » intentionnelles (nulls, doublons, montants négatifs) ✔️ Les données clients et agences sont cohérentes (IDs qui se recoupent) ✔️ Temps d'exécution < 30 secondes pour 100 000 lignes
Concepts clés	• Bibliothèque Faker : générer des données synthétiques réalistes (noms, IBAN, adresses) • Pandas DataFrame : structure de données tabulaire en mémoire, IO CSV/JSON • Distribution log-normale : modélise la répartition naturelle des montants financiers • Seed aléatoire (random.seed) : reproductibilité des données générées • Argparse : parser les arguments en ligne de commande en Python • Injection de bruit : simuler les erreurs réelles (valeurs nulles, doublons, outliers)

US.06 Script d'ingestion Bronze au format Parquet

Récit	En tant que **Dev** Je veux *créer un script d'ingestion qui déplace les fichiers sources vers landing_zone puis vers bronze au format Parquet* Afin de avoir une couche d'ingestion robuste, traçable et performante
Critères	✔️ Le script ingest_bronze.py lit depuis landing_zone/ et écrit dans bronze/ ✔️ Le format de sortie est Parquet avec compression Snappy ✔️ Les fichiers sont organisés par partition : bronze/year=YYYY/month=MM/day=DD/ ✔️ Un log d'ingestion est produit avec : fichier source, nombre de lignes, timestamp ✔️ En cas d'erreur, le fichier source est déplacé vers une zone quarantine/
Concepts clés	• Format Parquet : format colonnaire binaire — compression ~75% vs CSV,

	lecture ~10x plus rapide • PyArrow vs Pandas : PyArrow pour l'écriture Parquet native (plus performant) • Partitionnement Hive : organiser les fichiers par clés de partition pour filtrer sans tout lire • Landing zone : zone tampon entre la source et le stockage — principe des zones de staging • Gestion des erreurs Python : try/except, logging, dead-letter queue (zone quarantaine) • Checksum MD5 / SHA256 : vérifier l'intégrité d'un fichier lors du transfert

US.07 Ajouter des métadonnées techniques en Bronze	
Récit	En tant que **Data Engineer** Je veux *ajouter des colonnes techniques (date d'ingestion, nom du fichier source) dans la couche Bronze* Afin de assurer la traçabilité complète de chaque enregistrement, de la source à la Gold
Critères	✔ La colonne _ingestion_timestamp est ajoutée à chaque fichier Parquet Bronze ✔ La colonne _source_filename contient le nom du fichier d'origine ✔ La colonne _row_hash contient un hash MD5 de la ligne pour détecter les doublons exacts ✔ Ces colonnes sont documentées dans le dictionnaire de données (avec le préfixe _ pour les distinguer)
Concepts clés	• Colonnes techniques vs colonnes métier : convention _ ou __ pour distinguer les métadonnées • Hash de ligne : hash(concat(toutes les colonnes)) → identifiant unique d'un enregistrement • Audit trail : traçabilité obligatoire dans les systèmes bancaires (Basel III, SOX) • Schema evolution : comment gérer l'ajout de colonnes sans casser les pipelines existants • Timestamp UTC vs local : toujours stocker en UTC, convertir à l'affichage seulement

ÉPIC 3 — COUCHE SILVER | Nettoyage & Normalisation

Objectif : Transformer la donnée « sale » en donnée fiable, normalisée et conforme. La couche Silver est le cœur du pipeline : c'est ici que la qualité de données est établie. En milieu bancaire, une donnée erronée peut avoir des conséquences légales et financières majeures.

US.08 Nettoyer les données — Qualité de base

Récit	En tant que **Data Engineer** Je veux *nettoyer les données (supprimer les doublons, gérer les valeurs manquantes/nulles)* Afin de garantir que toute donnée qui entre dans Silver est fiable et utilisable
Critères	✔ Les doublons exacts (même _row_hash) sont supprimés avec journalisation du nombre ✔ Les enregistrements avec transaction_id null sont rejetés vers une table d'erreurs ✔ Les montants null sont soit imputés (0.00 si débit sans montant) soit rejetés selon la règle métier ✔ Un rapport de qualité est produit : X lignes en entrée → Y lignes rejetées → Z lignes en Silver ✔ Les règles de nettoyage sont documentées et versionnées
Concepts clés	• Déduplication : df.drop_duplicates() vs déduplication sur sous-ensemble de colonnes • Gestion des nulls : stratégies fill (imputation) / drop / flag (marqueur booléen is_null) • Data profiling : analyser une distribution avant de décider comment traiter les anomalies • Rejection table (dead-letter) : ne pas perdre les données rejetées, les stocker pour audit • Great Expectations / Pandera : frameworks Python de validation de schémas de données • SLA de qualité : définir un seuil acceptable (ex: max 1% de nulls sur montant)

US.09 Normaliser les formats de données

Récit	En tant que **Dev** Je veux *normaliser les formats (dates ISO 8601, montants en centimes, codes devises ISO 4217)* Afin de que toutes les sources parlent la même langue et que les calculs soient exacts
Critères	✔ Toutes les dates sont au format ISO 8601 : YYYY-MM-DDTHH:MM:SSZ (UTC) ✔ Tous les montants sont convertis en centimes (integer) — plus de flottants pour la monnaie ✔ Les codes devises respectent la norme ISO 4217 (EUR, USD, XAF...) ✔ Les IBANs sont validés structurellement (algorithme modulo 97)

	✔ Les codes agences sont normalisés en majuscules sur 8 caractères
Concepts clés	• Problème des flottants IEEE 754 : 0.1 + 0.2 ≠ 0.3 en binaire → toujours stocker en centimes (int) • ISO 8601 : standard international des dates — éviter les ambiguïtés jour/mois • Pytz / dateutil : manipulation des fuseaux horaires en Python • ISO 4217 : codes officiels des devises — le Franc CFA est XAF (zone CEMAC) • Validation d'IBAN : structure = code pays (2) + checksum (2) + BBAN (jusqu'à 30 chars) • Encoding UTF-8 : normaliser l'encodage des caractères spéciaux dès la Silver

US.10 Anonymiser les données clients — RGPD	
Récit	En tant que **Product Owner (Banque)** Je veux *anonymiser les données sensibles des clients (noms, emails, numéros de téléphone)* Afin de respecter le RGPD et protéger les données personnelles des clients de la banque
Critères	✔ Les champs PII (noms, prénoms, emails) sont remplacés par un hash SHA-256 + salt ✔ Les numéros de compte complets sont tronqués (seuls les 4 derniers chiffres visibles) ✔ Une table de correspondance chiffrée est stockée séparément avec accès restreint ✔ Les tests vérifient qu'aucune donnée nominative n'apparaît dans Silver ou Gold ✔ Un registre de traitement RGPD est produit
Concepts clés	• RGPD (Règlement Général sur la Protection des Données) : droits des personnes, obligations des entreprises • PII (Personally Identifiable Information) : noms, emails, téléphones, adresses IP... • Anonymisation vs pseudonymisation : différence légale et technique fondamentale • SHA-256 + salt : hash cryptographique + graine secrète pour éviter les rainbow tables • Tokenisation : remplacer une valeur sensible par un token opaque réversible • Data masking : masquer partiellement une valeur (ex: **** **** **** 1234 pour un CB) • Privacy by Design (Art. 25 RGPD) : intégrer la protection des données dès la conception

ÉPIC 4 — COUCHE GOLD | Business Logic & Analytics

Objectif : Créer de la valeur pour les analystes financiers. La couche Gold est la vitrine du pipeline. C'est ce que voient les métiers. Chaque table Gold répond à une question business précise et est optimisée pour la performance de lecture.

US.11 Calculer le solde total par client et par jour

Récit	En tant que **Analyste Financier** Je veux *calculer le solde total par client et par jour* Afin de suivre l'évolution quotidienne des dépôts et détecter des tendances
Critères	✔ La table gold.soldes_journaliers contient : client_id, date, solde_debut, total_credits, total_debits, solde_fin ✔ Le calcul est cumulatif : le solde du jour = solde_fin de la veille + mouvements du jour ✔ Les devises sont toutes converties en EUR (taux du jour stocké dans une table de référence) ✔ La table est partitionnée par date pour des requêtes rapides sur une période ✔ Un test vérifie que solde_fin = solde_debut + total_credits - total_debits
Concepts clés	• Fenêtres glissantes SQL / Pandas : fonctions window (LAG, SUM OVER PARTITION BY) • Cumulative sum : calcul de soldes progressifs sur une série temporelle • Taux de change : stocker les taux journaliers dans une table de dimension (dim_currency) • Partitionnement Parquet : diviser les données pour ne lire que ce qui est nécessaire • Slowly Changing Dimension (SCD) : comment gérer l'évolution du taux de change dans le temps

US.12 Détecter les transactions suspectes

Récit	En tant que **Product Owner / Équipe Conformité** Je veux *identifier automatiquement les transactions suspectes (montant > 10 000€, patterns anormaux)* Afin de aider l'équipe conformité à respecter les obligations LCB-FT (Lutte Contre le Blanchiment)
Critères	✔ La table gold.transactions_suspectes identifie toutes les transactions > 10 000€ ✔ Un score de suspicion est calculé (montant, fréquence, pays destinataire) ✔ Les transactions en provenance de pays à risque (liste GAFI) sont marquées ✔ Un rapport quotidien est généré en CSV pour l'équipe conformité ✔ Le système est documenté pour audit réglementaire
Concepts clés	• LCB-FT : Lutte Contre le Blanchiment et le Financement du Terrorisme — obligation légale bancaire

	• GAFI (Groupe d'Action Financière) : organisation internationale qui publie la liste des pays à risque • Règle des 10 000€ : obligation déclarative légale (TRACFIN en France) • Score de risque : modèle de scoring combinant plusieurs indicateurs (montant, fréquence, géographie) • Anomaly detection : approche statistique (Z-score, IQR) pour identifier les outliers • False positive management : trouver l'équilibre entre sensibilité et spécificité

US.13 Modéliser en schéma en étoile (Star Schema)	
Récit	En tant que **Data Architect** Je veux *modéliser la couche Gold avec une Table de Faits centrale et des Tables de Dimensions* Afin de optimiser les performances des requêtes analytiques et faciliter la consommation par les outils BI
Critères	✔ La table de faits fact_transactions contient les mesures numériques et les clés étrangères ✔ Les dimensions sont créées : dim_client, dim_agence, dim_date, dim_devise ✔ La dim_date contient des attributs calculés : jour_semaine, trimestre, est_ferie, est_weekend ✔ Les jointures entre fait et dimensions sont documentées avec un schéma visuel ✔ Les requêtes de validation tournent en < 5 secondes sur 30 jours de données
Concepts clés	• Schéma en étoile vs flocon de neige (Star vs Snowflake) : simplicité vs normalisation • Table de Faits : contient les mesures (montant, quantité) et les clés étrangères — elle est grande • Table de Dimensions : contient les attributs descriptifs — elle est petite mais riche • Surrogate key vs natural key : clé technique générée vs clé métier existante • Dimension Date : la dimension la plus utilisée en BI — précalculer les attributs temporels • OLAP cube : ce que les outils BI construisent implicitement à partir du schéma en étoile • Cardinalité des jointures : une transaction a UN client, UNE agence, UNE devise (N:1)

ÉPIC 5 — INDUSTRIALISATION & QUALITÉ | Du script manuel au système professionnel

Objectif : Passer d'un ensemble de scripts manuels à un système robuste, automatisé et documenté. Un pipeline qui ne tourne qu'à la main n'est pas un pipeline — c'est un script. L'industrialisation le transforme en un actif de l'entreprise.

US.14 Orchestrer le pipeline avec Airflow

Récit	En tant que **Dev** Je veux *utiliser Apache Airflow pour que le pipeline tourne automatiquement chaque matin à 6h00* Afin de éliminer les interventions manuelles et garantir la disponibilité des données pour les analystes en début de journée
Critères	✔ Un DAG Airflow bank_pipeline_daily est créé avec les tâches Bronze → Silver → Gold ✔ Le DAG se lance automatiquement à 6h00 chaque jour (schedule_interval='0 6 * * *') ✔ En cas d'échec d'une tâche, une alerte email est envoyée et les tâches aval sont bloquées ✔ La rétention est configurée : les logs sont conservés 30 jours ✔ Un tableau de bord Airflow permet de visualiser l'historique des exécutions
Concepts clés	• Apache Airflow : plateforme d'orchestration de workflows, DAG = Directed Acyclic Graph • DAG : graphe orienté acyclique — modélise les dépendances entre tâches sans cycle • Cron syntax : '0 6 * * *' = tous les jours à 6h00 — apprendre la notation cron • Backfill : rejouer des exécutions passées — indispensable lors d'une reprise après incident • SLA (Service Level Agreement) : s'engager sur une heure de disponibilité des données • Alternatives à Airflow : Prefect, Dagster, Luigi — choisir selon la maturité de l'équipe • Idempotence des DAGs : une exécution rejoué doit donner le même résultat

US.15 Data Quality Checks automatisés

Récit	En tant que **Scrum Master / Équipe** Je veux *mettre en place des tests de qualité automatiques qui bloquent le pipeline si une anomalie est détectée* Afin de garantir qu'aucune donnée erronée n'atteint la couche Gold et ne contamine les KPIs métier
Critères	✔ Un test vérifie qu'aucun montant négatif n'entre dans Gold (assertion Python) ✔ Un test vérifie que le nombre de transactions du jour est dans une plage normale (±30% vs moyenne) ✔ Un test vérifie l'unicité de transaction_id dans Silver

	✔ Les tests échoués bloquent automatiquement le passage à l'étape suivante ✔ Un rapport HTML de qualité est généré à chaque exécution
Concepts clés	• Test-driven data engineering : écrire les tests AVANT de coder les transformations • Pytest : framework de test Python — écrire des tests unitaires et d'intégration • Great Expectations : framework spécialisé pour les tests de données (expectations = contrats) • Circuit breaker : pattern qui stoppe un pipeline dégradé plutôt que de propager des erreurs • Data contract : accord formel sur le schéma et la qualité attendus entre producteur et consommateur • Monitoring vs testing : différence entre vérifier au moment de l'exécution vs en continu

US.16 Documenter le dictionnaire de données	
Récit	En tant que **Équipe (collectif)** Je veux *produire et maintenir un dictionnaire de données complet pour tous les futurs utilisateurs* Afin de garantir que n'importe qui peut comprendre et utiliser les données sans dépendre des créateurs du pipeline
Critères	✔ Un dictionnaire de données couvre les 3 couches : Bronze, Silver, Gold ✔ Chaque table contient : nom, description, propriétaire, fréquence de mise à jour ✔ Chaque colonne contient : nom, type, description, règle de calcul, exemple, valeurs possibles ✔ Un glossaire métier définit les termes bancaires utilisés (solde, virement, virement SEPA...) ✔ Le dictionnaire est versionné avec Git et mis à jour à chaque sprint
Concepts clés	• Data Catalog : outil centralisé de gouvernance des données (Apache Atlas, DataHub, Alation) • Lineage (lignage) : tracer le chemin d'une donnée de sa source jusqu'à son utilisation • Glossaire métier : définir les termes avec les experts métier — évite les malentendus • Documentation as Code : gérer la doc comme du code (versioning, review, CI/CD) • OpenAPI / JSON Schema : standards pour documenter les interfaces de données • DAMA-DMBOK : référentiel international de gestion des données — standard de l'industrie

4. Récapitulatif — Priorisation MoSCoW

La méthode MoSCoW (Must have, Should have, Could have, Won't have) permet de prioriser les User Stories en accord avec la valeur délivrée et les contraintes de temps. En formation, la priorisation est aussi un exercice pédagogique en soi.

ID	Titre	Épic	Priorité	Effort (j)
US.01	Comprendre l'architecture Medallion	Fondations	Must Have	0.5
US.02	Installer l'environnement	Fondations	Must Have	1
US.03	Comprendre ETL vs ELT	Fondations	Must Have	0.5
US.04	Définir le schéma des données	Fondations	Must Have	1
US.05	Générer les transactions simulées	Bronze	Must Have	2
US.06	Script d'ingestion Bronze/Parquet	Bronze	Must Have	2
US.07	Ajouter les métadonnées Bronze	Bronze	Should Have	1
US.08	Nettoyer les données Silver	Silver	Must Have	2
US.09	Normaliser les formats	Silver	Must Have	1.5
US.10	Anonymisation RGPD	Silver	Must Have	2
US.11	Soldes journaliers par client	Gold	Must Have	2
US.12	Détecter les transactions suspectes	Gold	Should Have	1.5
US.13	Star Schema (Faits + Dimensions)	Gold	Should Have	2
US.14	Orchestration Airflow	Industrialisation	Could Have	3
US.15	Data Quality Checks	Industrialisation	Should Have	2
US.16	Dictionnaire de données	Industrialisation	Should Have	1

Estimation de la charge totale

Épic	Nb de US	Effort estimé	Priorité globale
Épic 1 — Fondations	4	3 jours	Must Have
Épic 2 — Bronze	3	5 jours	Must Have
Épic 3 — Silver	3	5.5 jours	Must Have
Épic 4 — Gold	3	5.5 jours	Should Have
Épic 5 — Industrialisation	3	6 jours	Should Have
TOTAL	16	~25 jours	—

5. Glossaire technique & bancaire

Ce glossaire réunit les termes techniques et métier utilisés tout au long du projet. Il constitue également un livrable de la US.16 (dictionnaire de données).

Termes techniques Data Engineering

Terme	Définition
Parquet	Format de fichier colonnaire binaire, compressé. Standard du Big Data.
ETL	Extract-Transform-Load : paradigme de pipeline où la transformation précède le chargement.
ELT	Extract-Load-Transform : on charge d'abord, la transformation se fait dans le warehouse.
DAG	Directed Acyclic Graph : graphe orienté sans cycle, modélise les dépendances entre tâches.
Idempotence	Propriété d'une opération qui donne le même résultat si exécutée une ou plusieurs fois.
Partitionnement	Diviser les données en sous-ensembles physiques pour optimiser les lectures.
SCD	Slowly Changing Dimension : stratégie pour gérer l'évolution des tables de dimension.
Data Lineage	Traçabilité de l'origine et du parcours d'une donnée dans le système.
Data Contract	Accord formel sur le schéma et la qualité attendus entre producteur et consommateur.
Surrogate Key	Clé technique générée artificiellement (UUID, séquence) indépendante du métier.
Dead-letter queue	Zone de stockage pour les données en erreur — permet de les analyser sans les perdre.

Termes bancaires & réglementaires

Terme	Définition
IBAN	International Bank Account Number — numéro de compte bancaire standardisé (ISO 13616).
BIC/SWIFT	Bank Identifier Code — identifiant unique d'une institution bancaire dans les virements.
SEPA	Single Euro Payments Area — zone de paiement en euros harmonisée (38 pays).
LCB-FT	Lutte Contre le Blanchiment et le Financement du Terrorisme — obligation légale bancaire.

Terme	Définition
RGPD	Règlement Général sur la Protection des Données — règlement européen (GDPR en anglais).
PII	Personally Identifiable Information — données permettant d'identifier une personne.
TRACFIN	Traitement du renseignement et action contre les circuits financiers clandestins (France).
GAFI	Groupe d'Action Financière — organisation internationale anti-blanchiment.
XAF	Code ISO 4217 du Franc CFA BEAC — utilisé dans la zone CEMAC (Cameroun inclus).
Virement	Transfert de fonds entre deux comptes par ordre bancaire.
Contrepartie	L'autre partie dans une transaction financière (bénéficiaire ou émetteur).

Document produit à visée éducative — Projet Bank Data Pipeline Medallion

Architecture Medallion · Python · Parquet · Airflow · RGPD · Star Schema